Institute for Data Engineering
Module Big Data – Lab Exercises

Project Topic B – Real-Time Traffic Monitoring

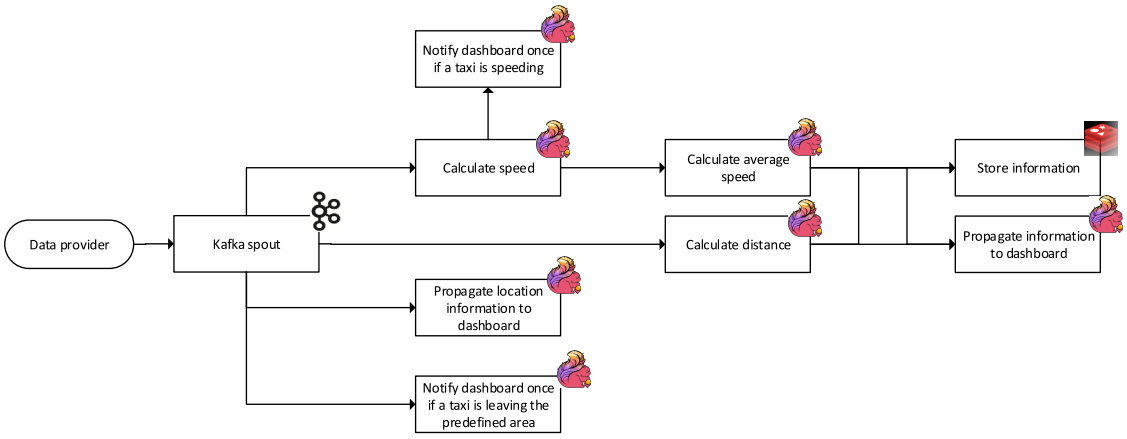

Figure 1: Stream Processing Topology

Introduction

Your task in this project is to realize a monitoring dashboard, which is updated in real-time based on streaming data. To provide information in real-time, the streaming data needs to be processed by a sequence of different stream processing operators, such as filters or aggregators [2]. These individual stream processing operators form a stream processing topology, as you can see in Figure 1.

The goal of this stream processing topology is to transform a series of geographical locations of different taxis [4][1] into value-added information that can be visualized on a dashboard to monitor the taxi fleet. In this task, you are going to combine several Big Data frameworks, such as Apache Flink[2], Apache Kafka[3], and Redis[4].

The preparatory task for this project is to populate an Apache Kafka instance with the location information of the taxis. The location data set contains the location information of different taxis and your task is to combine this information, to replay the location information according to their timestamps, and to submit the replayed data stream to Apache Kafka. Apache Kafka acts then as a data source for Apache Flink, which processes the location information to obtain the value-added information and forwards the value-added information to the dashboard.

The processing operators of this stream processing topology are implemented by several stream processing operators, which are described in detail below:

- The **Calculate speed** operator calculates the speed between two successive locations for each taxi, whereas the distance between two locations can be derived by the Haversine formula[5]. This operator represents a stateful operator because it is required to always remember the last location of the taxi to calculate the current speed [1].

[1] `https://github.com/roryhr/taxi-trajectories?tab=readme-ov-file`
[2] `https://flink.apache.org/`
[3] `http://kafka.apache.org`
[4] `http://redis.io`
[5] `http://rosettacode.org/wiki/Haversine_formula`

- The **Calculate average speed** operator aggregates all speed information from the previous stream processing operator and calculates the average speed of all single speed information for a specific taxi. You may use this formula for calculating the average speed:

$$S_{avg} = \frac{1}{S_n} * \sum_{i=1}^{S_n} S_i \tag{1}$$

  S_n represents the number of all individual speed information and S_i represents the individual speed information. This operator also represents a stateful operator, since it has to collect all individual speed information to calculate the average speed for a taxi.

- The **Calculate distance** operator calculates the overall distance for each taxi, whereas you can also use the Haversine formula.

- The **Store information** operator stores the average speed for each individual taxi and the distance of the taxi ride so far to a Redis data structure store.

- The **Propagate information to dashboard** operator calculates the amount of all currently driving taxis, aggregates the overall distance covered by all taxis and forwards the data to the dashboard.

- The **Propagate location information to dashboard** operator forwards the location information of each taxi every five seconds to the dashboard, i.e., all location information in between are not visualized in the dashboard.

- The **Notify dashboard once if taxi is leaving the predefined area** and the **Notify dashboard once if taxi is speeding** operators analyze the speed respectively location of each taxi and whenever a taxi is faster than a predefined speed limit, i.e., 50 km/h, or leaves a predefined area, this operator forwards the information to the dashboard. The dashboard should issue a warning whenever a taxi is leaving the area with a radius of 10 km around the Forbidden City in Beijing. The dashboard should discard any information about taxis who leave the area with a radius of 15 km around the Forbidden City in Beijing.

Outcomes

The expected outcomes of this project are two-fold: (1) the actual project solution, (2) an intermediary and a final presentation of your results.

Project Planning

The initial draft architecture of the pipeline in pdf format (Use naming convention: `archi_⟨Team number⟩_⟨topic⟩e.g., archi_M4_A`) and the work packages distributed among each member of the team over the project timeline as a Gantt chart in pdf format (Use naming convention: `gantt_⟨Team number⟩_⟨topic⟩e.g., gantt_M4_A`) should be created and uploaded to your team's folder on Stud.IP, until **May 8, 2026**.

Project Solution

The project has to be hosted on a Git repository in TUHH's GitLab instance[6] – you will get instructions about the repository setup at the lab kickoff meeting. Every member of your team has to use its own, separate Git account. *We will check who has contributed to the source code*, so please make sure you use your own account when submitting code to the repository. We do not accept excuses like "We were coding together and used one account to submit the code". Students

[6] `https://collaborating.tuhh.de/`

have failed this module because of this in the past, and we would really like to avoid this in the future. Also, do not delete or hide old branches of the code. All implemented code (including branches) should be handed in.

Furthermore, it is required to provide an easy-to-follow README that details how to deploy, start and test the solution. The best practice is to provide a README that describes "Plug-and-Play" instructions on how to use your solution.

For the submission (see below), you also have to create one or more Docker images, which contain(s) your complete implemented solution, i.e., including all dependencies.

Presentations

There are two presentations. The first one is during the consultation hours, and describes your status at that particular point of time. This presentation will not be graded, i.e., it is primarily a way to check your progress and to show to other groups what you are working on and how you want to solve the problem.

The second presentation is during the final meetings and contains all your results. The actual dates are announced in Stud.IP.

Every member of your team is required to present in either the first *or* the second presentation. Each presentation needs to consist of a slides part and a demo of your implementation. Think carefully about how you are going to demonstrate your implementation, as this will be part of the grading. You have 10 minutes (strict) of time for your presentation in the consultation hour, and 15 minutes (also strict) of time for your presentation at the final meeting. At the first presentation, you can go for a slide-based presentation, a live demo, or a combination of both, depending on your progress until that point of time. The demo is by far the most important part of the presentation, so having a 2-3 minutes demo is not sufficient.

All topics deliver enough content to fill the 10 and 15 minutes of the presentations. If you are only able to fill, e.g., half of the time, this most likely means that you are missing something. In this case, contact your tutor early enough, i.e., not two days before your presentation is due.

The past has shown that providing a nice use case story usually helps to present the project outcomes. While providing such a use case story is not an absolute must, it will surely help the audience to understand your work better.

Grading

A maximum of 60 points are awarded in total for the project. Of this, 70% are awarded for the implementation (taking into account both quality and creativity of the solution as well as code quality and documentation), and 30% are awarded for the final presentation (taking into account content, quality of slides, presentation skills, and discussion).

A strict policy is applied regarding plagiarism. Plagiarism in the source code will lead to 0 points for the particular student who has implemented this part of the code. If more than one group member plagiarizes, this may lead to further penalties, i.e., 0 points for the implementation of the whole group.

Deadline

The hard deadline for the project will be announced via Stud.IP. Please upload your presentation to your team's folder on Stud.IP (Use the naming convention: `final_⟨Team number⟩_⟨topic⟩e.g., final_M4_A`). The Docker images need to be uploaded to DockerHub (use your own private DockerHub accounts for this) and made available to the lecturer team (push the images to the DockerHub as public images). For this, please write in the README on how to access the containers. The README should be uploaded to the Git repository. The deadline for this is also the same as the project submissions deadline. Late submissions will not be accepted.

Test Cloud Infrastructure

You can use any public cloud infrastructure, including the one that provide free credits for the students (e.g., Azure, GCP, AWS, etc.). Further information will be provided via Stud.IP (under: General Information → Lab Exercises).

For the final marking of the cloud deployed solutions, you would be asked to keep you solutions running for a day. The exact date for each group will be notified via email or Stud.IP. Your cloud solution should be able to be accessed simply via a public URL. Indicate the URL in your readme file.

Stage 1

Your task in Stage 1 is to setup and configure Apache Kafka as well as Apache Flink, whereas it is sufficient to run them in local mode in this stage.

Furthermore, you have to implement the data provider to fill Apache Kafka with data. Therefore you may need to preprocess the data provided by the *T-drive* data set[7].

The data in the text files are structured as follows: The *taxiId* represents a unique identifier for one taxi across all locations, the *timestamp* represents the time when the location was recorded and *latitude* respectively *longitude* describe the geographical location. It is advisable that the data provider emits an end token as soon as the data for one taxi, i.e., one data file, has been submitted to the system, to signal that this taxi has stopped and does not emit any further information.

When handling Big Data in the real world, it is quite often the case that the data is processed as they are streamed, with minimal permanent storing, due to an infinite amount of data being streamed at a rapid pace from the data sources. The above steps want you to create a dataset which, later using Kafka, could emulate a similar infinitely real-time streaming data source, rather than a classical database with a constant set of data being stored statically. Any additional data processing beyond this purpose should be done during the data streaming phase (i.e., using Flink).

Tasks:

1. Setup and configure Apache Kafka in a Docker container.
2. Implement the data provider, which represents a producer for Apache Kafka. This data provider retrieves the location information from text files or an SQL database and submits it to Apache Kafka. The data needs to be replayed in the same order as it was recorded, i.e., all location information with the same timestamp needs to be submitted at the same time. It is advisable to make the submission speed configurable to apply different submission speeds during testing and actual performance tests of the system.
3. Setup and configure Apache Flink in a Docker container.
   - You may use Flink's *Apache Kafka Connector* to feed real-time data into your Flink topology.
   - Implement the required operators for the topology.

Please note: Stage 1 is what you should have achieved roundabout half-way through the semester. This is a recommendation, not a must. But it will help to avoid too much crunch time at the end of the semester.

Stage 2

In Stage 2, you have to implement the dashboard (see Figure 2), which retrieves the data from the stream processing topology and visualizes it for the user. You further need to test the performance

[7] `https://www.microsoft.com/en-us/research/publication/t-drive-trajectory-data-sample/`

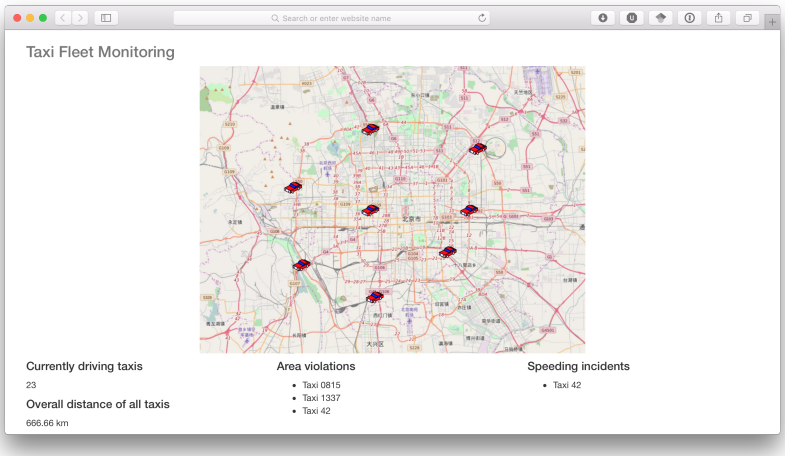

Figure 2: Taxi Fleet Monitoring Dashboard

of your implementation, by determining the maximal submission speed your implementation can handle. [8]

The outcome will then be compared with those from other groups. To benchmark the optimization results against those of other groups, it is required that the topology can be deployed on an Apache Flink cluster, e.g., hosted in the cloud.

Tasks:

1. Implement a basic Web-based GUI, featuring all the elements in Figure 2.
2. Retrieve the processed information from the stream processing topology and display the data in real-time, i.e., only show those taxis in the map which are currently driving within the specified areas and add a new incident for every violation retrieved from the stream processing topology.
3. Optimize your implementation to achieve a high throughput while maintaining valid results. For optimization ideas, you may have a look at the publication by Hirzel et al. [3].

[8] This basically requires you to determine the maximum throughput of your pipeline, i.e., the maximum amount of load that could be transferred at the highest speed along your pipeline.

You could define the pipeline this way; pipeline = from source → processing → to sink,
where,
source = the point of data ingestion into a Kafka topic via the producer,
sink = the point where the consumer receives the stream data after it has been processed. For instance, if you are caching the processed data in a database before reading it from the database to display in the Web UI, the database would be the sink. This means, you can ignore the parts before the source, which could include reading the data from the pre-processed dataset, and after the sink, which includes the delay your Web UI framework causes when reading from the database for example.

For this, a quantitative representation similar to a graph could be suggested, which represents,

- The end-to-end delay $T_2 - T_1$ of the pipeline, i.e., the difference between the timestamps when a data entry is ingested via the producer (e.g., the data entry `'2,2008-02-02 13:59:00,116.4302,39.91176'` is ingested at time T_1) and the timestamp that this data entry is processed and sent to the sink (the mentioned data entry is utilized to calculate the current speed, and this current speed value is then cached to the database at time T_2),
- How would this delay vary when you increase the load on the pipeline by increasing the number of taxis processed? This means: How does $T_2 - T_1$ vary with the number of taxis simultaneously processed?

Please refer to [1, 2, 3, 4], to get a better understanding of what to do. You don't need to read through everything, but have a look at the graphs which are used to represent the performances.

4. Document your applied stream processing optimizations and configurations for Apache Flink and their effects on the throughput.

References

[1] Raul Castro Fernandez, Matteo Migliavacca, Evangelia Kalyvianaki, and Peter Pietzuch. Integrating scale out and fault tolerance in stream processing using operator state management. In *2013 ACM SIGMOD International Conference on Management of Data*, pages 725–736, 2013.

[2] Buğra Gedik, Henrique Andrade, Kun-Lung Wu, Philip S Yu, and Myungcheol Doo. SPADE: The System S Declarative Stream Processing Engine. In *2008 ACM SIGMOD International Conference on Management of Data*, pages 1123–1134, 2008.

[3] Martin Hirzel, Robert Soulé, Scott Schneider, Buğra Gedik, and Robert Grimm. A catalog of stream processing optimizations. *ACM Computing Surveys*, 46(4):46, 2014.

[4] Jing Yuan, Yu Zheng, Chengyang Zhang, Wenlei Xie, Xing Xie, Guangzhong Sun, and Yan Huang. T-drive: Driving directions based on taxi trajectories. In *18th SIGSPATIAL International Conference on Advances in Geographic Information Systems*, pages 99–108, 2010.